CA3

by Brage Bestvold and Lasse Lindholm

Imports

In [ ]:
```
# for preprocessing
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# different classifers
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# for measuring performance
from sklearn.metrics import accuracy_score



# To stop warnings from
import warnings
warnings.filterwarnings("ignore", category=Warning, module="sklearn")
```

Reading data

In [3]:
```
train = pd.read_csv("train.csv")
train.head()
```

Out[3]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	a
0	7.8	0.610	0.29	1.6	0.114	9.0	29.0	0.99740	3.26	1.56	
1	8.4	1.035	0.15	6.0	0.073	11.0	54.0	0.99900	3.37	0.49	
2	8.1	0.290	0.36	2.2	0.048	35.0	53.0	0.99500	3.27	1.01	
3	5.2	0.340	0.00	1.8	0.050	27.0	63.0	0.99160	3.68	0.79	
4	15.6	0.645	0.49	4.2	0.095	10.0	23.0	1.00315	2.92	0.74	

Data exploration and visualisation

Overview of the training data

The output below shows the number of rows and columns, missing values, how balanced the two quality classes are (`quality` : 0 = lower quality, 1 = higher quality), and summary statistics (mean, std, min, quartiles, max) for every column. `Id` is only a row identifier and carries no information about wine quality.

In [4]:
```
# Size of the data, summary statistics and class balance
print("Train shape (rows, columns):", train.shape)
print("How much of each quality"+"\n", train["quality"].value_counts(normalize=True
train.describe().T # Transposed table
```

```
Train shape (rows, columns): (835, 13)
How much of each quality
 quality
1    0.500599
0    0.499401
Name: proportion, dtype: float64
```

Out[4]:

	count	mean	std	min	25%	50%	75%	m
fixed acidity	835.0	8.310898	1.757359	4.60000	7.1000	7.90000	9.1000	15.90(
volatile acidity	835.0	0.540042	0.179786	0.12000	0.4100	0.54000	0.6475	1.58(
citric acid	835.0	0.260790	0.197739	0.00000	0.0800	0.24000	0.4100	1.00(
residual sugar	835.0	2.542994	1.420451	0.90000	1.9000	2.20000	2.6000	15.50(
chlorides	835.0	0.088674	0.051485	0.01200	0.0710	0.08000	0.0910	0.61
free sulfur dioxide	835.0	15.859880	10.444733	1.00000	7.0000	13.00000	21.0000	68.00(
total sulfur dioxide	835.0	47.077246	33.271056	7.00000	22.0000	38.00000	63.0000	289.00(
density	835.0	0.996799	0.001946	0.99007	0.9956	0.99675	0.9979	1.003
pH	835.0	3.308707	0.159072	2.74000	3.2000	3.31000	3.4000	4.01(
sulphates	835.0	0.658048	0.179891	0.33000	0.5500	0.61000	0.7250	2.00(
alcohol	835.0	10.396287	1.087407	8.40000	9.5000	10.10000	11.0000	14.90(
quality	835.0	0.500599	0.500299	0.00000	0.0000	1.00000	1.0000	1.00(
Id	835.0	800.470659	468.873363	0.00000	408.0000	784.00000	1225.5000	1597.00(

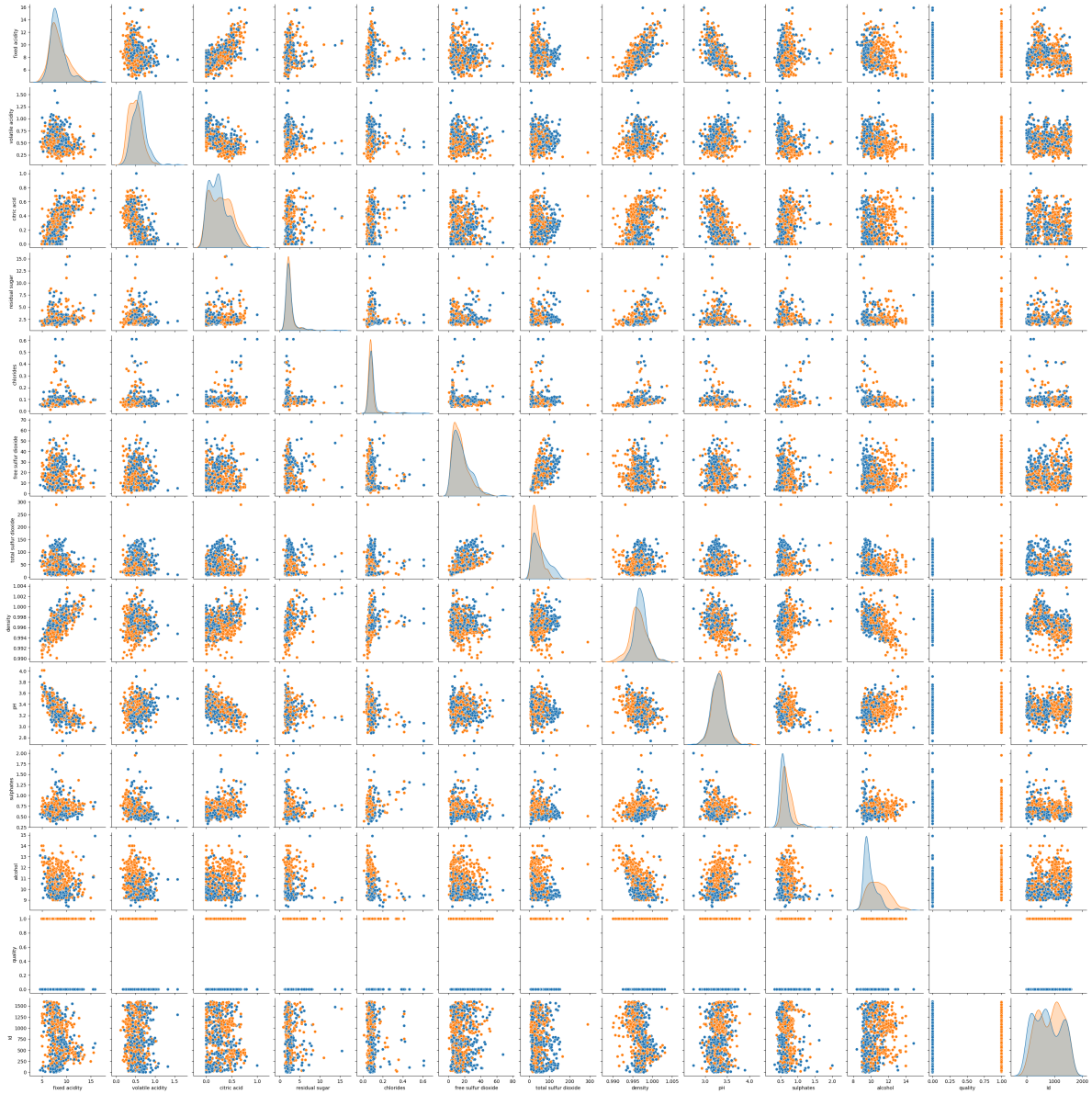
Pairplot

Compares all 13 characteristics to every other characteristic making a 13x13 grid of small plots. Each small plot compared a feature named at the bottom of its column (x-axis) with a feature named to the left of the row (y-axis). Points are colors by quality where orange means 1, blue means 0.

on the diagonal, features are compared to itself, so seabord shows a distribution graph for each quality. Above and below the diagonal, the plots are mirrored since they each feature compare to each other in oposite order.

In [5]:
```
sns.pairplot(train, vars=train.keys(), hue="quality")

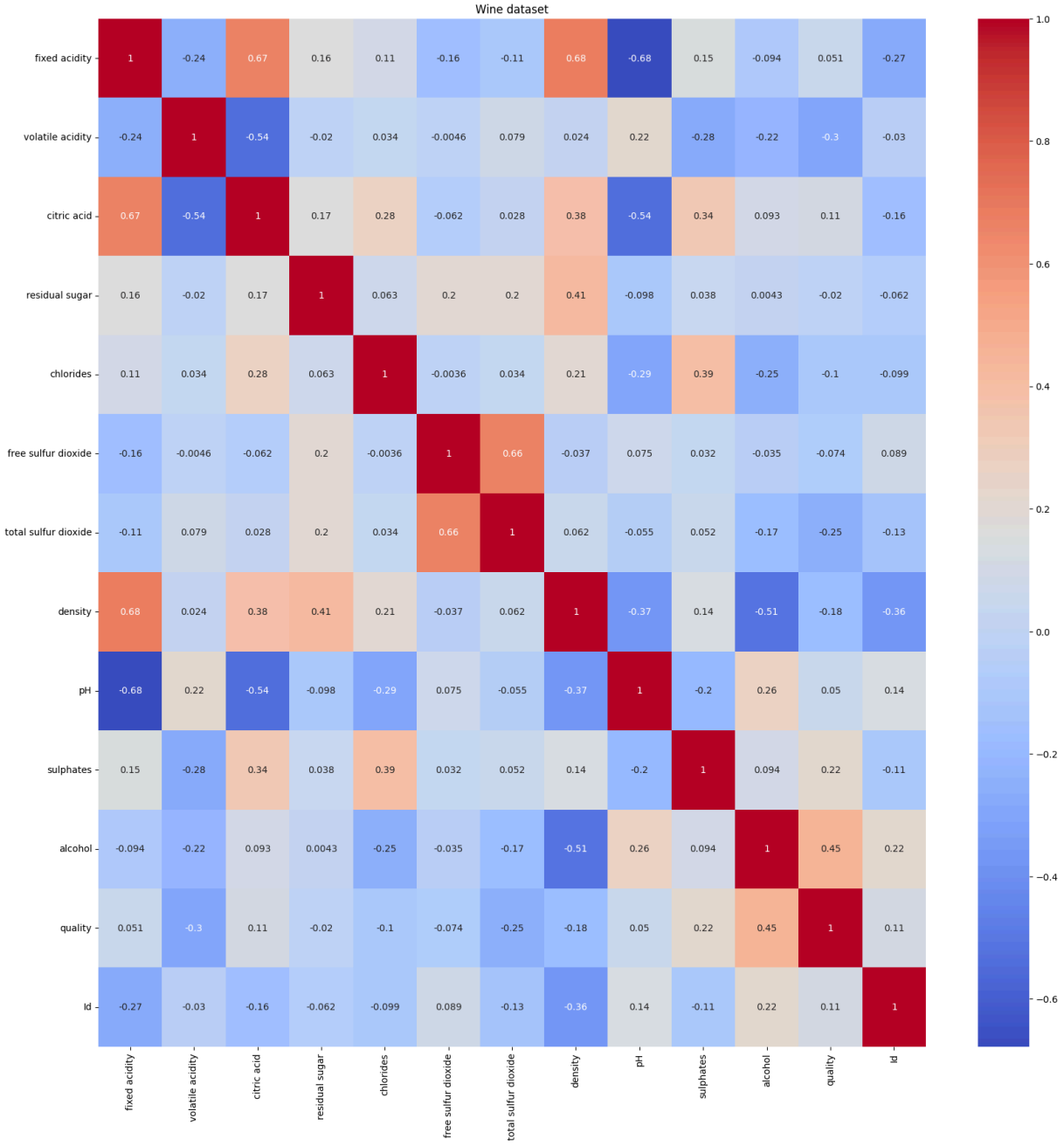
plt.show()
```



In [6]:

```
# Modified correlation heatmap taken from the 04_correlationmap.ipynb example from
# https://nmbu.instructure.com/courses/14056/files/folder/Lecture_12_Preprocessing?
def plot_correlation_heatmap(data, title):
    """
    Plot a correlation heatmap for a given dataset
    """
    df = pd.DataFrame(data, columns=data.keys())
    df['quality'] = data.quality
    sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
    plt.title(title)
    plt.show()

plt.figure(figsize=(20, 20))
plot_correlation_heatmap(train, 'Wine dataset')
```



Based on the pairplot and the correlation heatmap, we conclude that no features are reduntant, and thus we use all features when training models

Boxplots

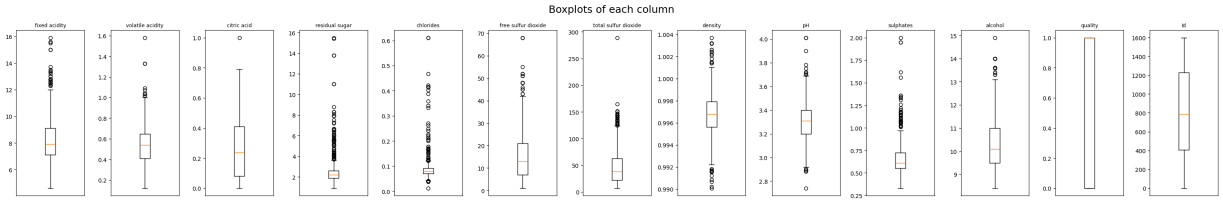
One boxplot per column, the box plot covers the middle 50% of values, the circles outside the ending lines are potential outliers, and the orange line is the median value.

In [7]:
```
fig, ax = plt.subplots(1, len(train.columns), figsize=(30, 5))
fig.suptitle("Boxplots of each column", fontsize=18)

for i, col in enumerate(train.columns):
    ax[i].boxplot(train[col])
    ax[i].set_title(col, fontsize=9)
```

```
    ax[i].set_xticks([])

plt.tight_layout()
plt.show()
```



Based on the boxplot the data will benefit from scaling

Data cleaning

In [8]:
```
print(train.shape)
train = train.dropna()
print(train.shape)
print("No rows with missing data")
```

```
(835, 13)
(835, 13)
No rows with missing data
```

Finding best model

We decide not to train a perceptron model since it trains very slowly and performs poorly on datasets that are not linearly seperable. We also drop decisionTreeClassifier since randomForeest is multiple decisiontrees, so it is very unlikely that a single tree will perform better.

In [16]:
```
num_seeds = 10
X = train.drop(axis=1, labels=["quality", "Id"])
y = train["quality"]

scaler = StandardScaler()


classifier_types = [LogisticRegression, SVC, RandomForestClassifier, KNeighborsClas
classifiers = []

for seed in range(num_seeds):
    for classifier in classifier_types:
        if classifier in (LogisticRegression, SVC):
            for c in range(-8,10,1):
                classifiers.append((classifier(max_iter=100, C = 10**c, random_stat
        elif classifier == KNeighborsClassifier:
            for n_neighbors in range(1,13):
                classifiers.append((classifier(n_neighbors=n_neighbors), classifier
        else:
            for depth in range(1,8):
                classifiers.append((classifier(n_estimators=1000, max_depth=depth,
```

```
val_scores = np.zeros(num_seeds)
train_scores = np.zeros(num_seeds)

best_val_score = 0
corr_train_score = 0
best_model = None
for id, (classifier, classifier_type) in enumerate(classifiers):
    for seed in range(num_seeds):
        X_train, X_val, y_train, y_val = train_test_split(X, y, test_size = 0.3, ra

        if classifier_type == RandomForestClassifier:
            X_train_scaled = X_train
            X_val_scaled = X_val
        else:
            X_train_scaled = scaler.fit_transform(X_train)
            X_val_scaled = scaler.transform(X_val)


        classifier.fit(X_train_scaled, y_train)
        y_pred_train = classifier.predict(X_train_scaled)
        y_pred = classifier.predict(X_val_scaled)

        train_scores[seed] = accuracy_score(y_train, y_pred_train)
        val_scores[seed] = accuracy_score(y_val, y_pred)

    mean_val = val_scores.mean()
    mean_train = train_scores.mean()
    # Update best score, check difference between validation score and traiing scor
    if (mean_val > best_val_score) and (abs(mean_val - mean_train) < 0.1):
        print(f"New Highscore: {mean_val}")
        best_val_score = mean_val
        corr_train_score = mean_train
        best_model = classifier
print(f"Best model: {best_model} \n Validation Score: {best_val_score} \n Train sco
```

```
New Highscore: 0.4705179282868526
New Highscore: 0.5215139442231076
New Highscore: 0.7115537848605579
New Highscore: 0.7422310756972113
New Highscore: 0.751394422310757
New Highscore: 0.751792828685259
New Highscore: 0.7669322709163346
New Highscore: 0.7681274900398407
New Highscore: 0.7689243027888446
New Highscore: 0.7713147410358566
New Highscore: 0.7717131474103586
Best model: RandomForestClassifier(max_depth=5, n_estimators=1000, random_state=5)
 Validation Score: 0.7717131474103586
 Train score: 0.8606164383561644
```

Fitting and testing best model

In [12]:
```
test = pd.read_csv("test.csv")
test = test.drop(axis=1, labels=["Id"])

best_model.fit(X, y)

y_pred = best_model.predict(test)
submission = pd.DataFrame()
submission["Id"] = range(len(y_pred))

submission["quality"] = y_pred
```

Kaggle submission

In [ ]:
```
submission.to_csv("submission.csv", index=False)
```